令和 8 年度　修士論文

LiDAR と不可視地図を用いたロバストな自己位置推定の開発

2026 年 12 月 24 日

宇都宮大学大学院
地域創生科学研究科
工農総合科学専攻

学籍番号：256765C
氏名：七久保　堅信

指導教員
尾崎 功一 教授
ミヤグスク レナート 准教授
田畑 研太 助教

目次

第 1 章　緒言　1

1.1　研究背景 . . . 1

1.2　本研究の目的 . . . 2

1.3　本論文の構成 . . . 2

第 2 章　関連研究　4

2.1　ロボットに関する従来研究 . . . 4

2.1.1　アプローチ A-1 . . . 4

2.1.2　アプローチ A-2 . . . 4

2.2　△△に関する従来研究 . . . 4

2.2.1　アプローチ B-1 . . . 5

2.2.2　アプローチ B-2 . . . 5

2.3　本研究の位置付け . . . 5

第 3 章　ハードウェア設計　6

3.1　システム構成 . . . 6

3.2　機構設計 . . . 6

3.2.1　移動機構の設計 . . . 7

3.2.2　マニピュレータの設計 . . . 7

第 4 章　Wi-Fi と LiDAR を用いた大域的自己位置推定　8

4.1　はじめに . . . 8

4.2　ハードウェア構成 . . . 8

4.2.1　Wi-Fi センサ . . . 8

4.3　自己位置推定に用いる地図 . . . 9

4.3.1　幾何地図 . . . 9

4.3.2　Wi-Fi 地図 . . . 10

4.4　LiDAR を用いた局所的自己位置推定 . . . 11

4.4.1　動作モデルに基づいたパーティクルの更新 . . . 12

4.4.2　尤度計算 . . . 14

4.4.3　位置推定 . . . 15

4.4.4　パーティクルのリサンプリング . . . 15

第 5 章　ソフトウェア設計　16

5.1 カルマンフィルタの実装 16
5.2 ソフトウェア評価 16

第 6 章 実環境におけるシステム評価 18
6.1 実験目的 18
6.2 実験環境 18
6.3 実験結果 18
6.3.1 現在の位置姿勢 18
6.3.2 次の位置姿勢 18
6.3.3 更新 18
6.3.4 2 台の 3D-LiDAR 間の位置姿勢補正 19
6.4 考察 20

第 7 章 シミュレーション評価 21
7.1 シミュレーション環境の構築 21
7.2 実験概要 21
7.3 考察 21

第 8 章 結言 22
8.1 結論 22
8.2 課題 22
8.3 展望 22

謝辞 23

付録 A 詳細な電子回路図 25

付録 B 主要なプログラムのソースコード 26
B.1 C++ 26
B.2 Python 26

付録 C 追加実験 27
C.1 実験 A 27
C.2 実験 B 27

図目次

3.1 システム構成 · 6

3.2 自律移動ロボット · 7

    3.2(a) 移動機構 · 7

    3.2(b) マニピュレータ · 7

4.1 WiFi sensor · 8

4.2 Geometric map created by gmapping · 11

4.3 Map converted from geometric map to occupied grid map · · · · · · · · · · · · · 11

4.4 Overview of wireless-based localization using signal strength mapping[**miyagusuku2018precise**] 12

4.5 Robot Pose Shown in a Global Coordinate System[**fox2005probabilistic**] · · · · 14

4.6 Odometry model: The robot motion in the time interval$(t-1, t)$ is approximated by a rotation δ_{rot1}, followed by a translation δ_{trans} and a second rotation δ_{rot2}. The turns and translation are noisy[**fox2005probabilistic**]. · · · · · · · · · · · · · · · · · · 14

A.1 システム全体の回路図 · 25

表目次

4.1 Robot specification 9
4.2 Specification of WiFi sensor 10
6.1 ICP によって算出された補正量 19
6.2 ICP 補正量の統計値 19
6.3 LiDAR 2 に適用する固定補正値 19
6.4 固定補正適用後の ICP 残差 20
6.5 固定補正後の残差 20
7.1 性能比較 21

第 1 章

緒言

1.1　研究背景

近年，日本では少子高齢化に伴う労働力不足が深刻な社会問題となっている．日本の生産年齢人口は，2020 年の約 7,500 万人から，2040 年には約 6,200 万人まで減少することが見込まれており [1], 人手不足が経済成長の制約となって深刻な影響を与えることが懸念されている．

近年，少子高齢化に伴う労働力不足が深刻化しており，労働力の代替手段として自律移動ロボットの導入が期待されている．自律移動ロボットは，荷物の運搬や警備，道案内などの様々な用途で活用が進められており，今後は屋内だけでなく屋外を含む多様な環境での社会実装が期待されている．しかし，実環境では人や車両などの動的障害物の存在や，周囲の環境が変化することにより，自律移動が困難になる場合がある．自律移動ロボットが目的地まで安定して走行するためには，周囲の環境を認識し，ロボット自身の位置および姿勢を正確に推定する必要がある．そのため，自律移動ロボットの社会実装に向けて，屋内外において環境変化に左右されないロバストな自己位置推定の確立が必要不可欠である．

自己位置推定の手法として，LiDAR やカメラなどの幾何情報に基づく位置推定の手法や，衛星電波を利用した GPS による位置推定の手法が挙げられる．LiDAR を用いた自己位置推定では，レーザ光により周囲の物体までの距離を計測し，事前に作成した幾何地図と観測した距離情報を照合することで，ロボットの位置および姿勢を推定する．また，カメラを用いた自己位置推定では，撮影した画像から周囲の物体の特徴を抽出し，画像間の特徴や事前に作成した地図と照合することで自己位置推定を行う．これらの手法は，周囲の環境から十分な特徴が得られる場合には，高精度な自己位置推定が可能である．しかし，LiDAR は人混みや車両などの動的障害物によって観測した幾何情報と事前に作成した地図との間に不一致が生じる場合がある．また，周囲に類似した幾何構造が多い環境や，自己位置推定に必要な特徴が少ない環境では，誤った位置に収束する可能性がある．一方，カメラは照明条件の変化や逆光，周囲の物体による遮蔽などにより画像から十分な特徴を抽出できず，自己位置推定精度が低下する可能性がある．さらに，GPS は屋外環境において広範囲で位置情報を取得できるものの，建物などによるマルチパスによって測位精度が低下する場合や，屋内では衛星電波を十分に受信できない問題がある．このため，これらのセンサのみでは，あらゆる環境において安定した自己位置推定を実現することは困難である．

そこで本研究室では，磁気強度や Wi-Fi 電波強度情報を用いた「不可視地図」に基づく自己位置推定手法を研究している．不可視地図とは，環境中に存在する磁気強度や Wi-Fi 電

波強度などの目に見えない情報を，位置情報と対応付けて記録した地図である．磁気強度を用いた自己位置推定では，環境中に存在する構造物や磁性体などの影響によって生じる磁気強度の分布を利用する．事前に観測した磁気強度をガウス過程回帰（Gaussian Process Regression：GPR）によって補間することで磁気地図を構築し，ロボットに搭載した磁気センサの観測値と磁気地図を照合することで自己位置推定を行う．一方，Wi-Fi 電波強度を用いた自己位置推定では，環境中に存在するアクセスポイント（Access Point：AP）から受信した電波強度を利用する．各 AP から受信した Wi-Fi 電波強度を GPR によって補間した Wi-Fi 地図を構築し，観測した Wi-Fi 電波強度と地図上の電波強度を照合することで自己位置推定を行う．

不可視地図に基づく自己位置推定手法は，LiDAR などの幾何情報に基づく自己位置推定手法と比較して精度が劣る場合があるものの，人混み等の幾何情報の変化による影響を受けにくい特徴がある．しかし，磁気強度や Wi-Fi 電波強度は環境によって分布が異なり，観測した情報だけでは位置を十分に識別できない場合もある．そのため，不可視地図に基づく自己位置推定のみでは，常に高精度な自己位置推定を実現することは困難である．一方，LiDAR による自己位置推定は幾何情報が安定している環境では高精度な自己位置推定が可能であることから，これらの異なる特徴を持つセンサを組み合わせることで，互いの弱点を補完できると考えられる．

本研究では，LiDAR と不可視地図に基づく自己位置推定手法を統合することで，幾何情報が変化する環境においてもロバストな自己位置推定の実現を目指す．さらに，LiDAR による自己位置推定が失敗した場合には，他の自己位置推定手法を利用することで，安定した自己位置推定の継続および失敗した自己位置推定の復帰を目指す．

本論文では，磁気強度および Wi-Fi 電波強度を用いた不可視地図に基づく自己位置推定手法を構築する．また，LiDAR と不可視地図を統合した自己位置推定手法を構築し，幾何情報が変化する環境における有効性を検証する．さらに，LiDAR MCL，磁気強度に基づく Mag MCL，Wi-Fi 電波強度に基づく Wi-Fi MCL，およびこれらのセンサ情報を統合した Multi MCL を並列に動作させ，各手法の位置推定状態に応じて適応的に自己位置推定手法を切り替えるシステムを構築する．最後に，他の自己位置推定手法のパーティクル分布を利用することで，位置推定が失敗した手法を復帰させるシステムを構築し，その有効性を検証する．

1.2　本研究の目的

研究目的を書きます．

1.3　本論文の構成

論文構成を書きます．以下のように各章を参照します．

第2章では，かか．

第3章では，きき．

第5章では，くくく．

第6章では，けけけ．

第7章では，こここ．

第8章では，わわわ．

第 2 章

関連研究

2.1　ロボットに関する従来研究

ロボットに関する研究．ああ．

2.1.1　アプローチ A-1

尾崎らは，こんなアプローチを提案した [2]．いいい．

2.1.2　アプローチ A-2

他には，こんなアプローチがある [3, 4]．うう．

2.2　△△に関する従来研究

△△に関する研究．かかか.

2.2.1　アプローチ B-1

きき.

2.2.2　アプローチ B-2

くく.

2.3　本研究の位置付け

本研究の位置付けを書きます．ささ.

第 3 章

ハードウェア設計

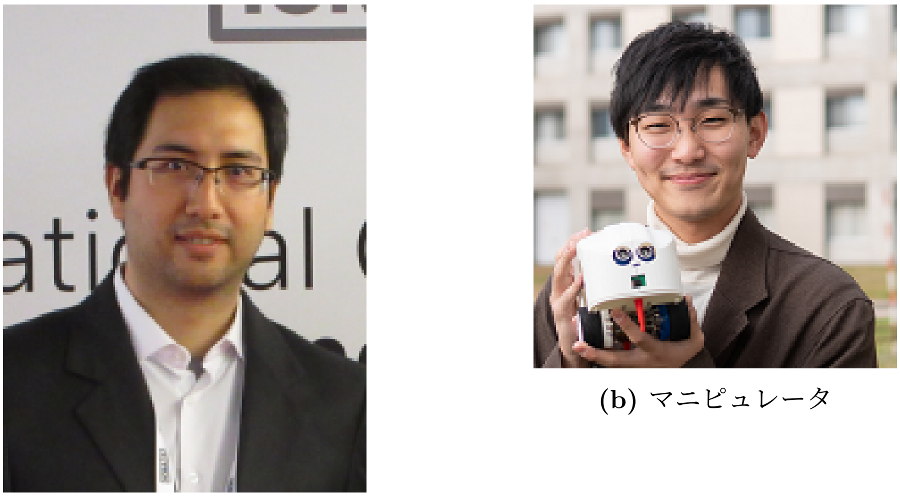
3.1　システム構成

システム構成は 3.1 です．ああ

Fig. 3.1 システム構成

3.2　機構設計

移動機構とマニピュレータは 3.2 です．かかか



(b) マニピュレータ

(a) 移動機構

Fig. 3.2 自律移動ロボット

3.2.1　移動機構の設計

3.2a を説明します．きき

3.2.2　マニピュレータの設計

3.2b を説明します．くくく

第 4 章

Wi-Fi と LiDAR を用いた大域的自己位置推定

4.1　はじめに

本章では，Wi-Fi 尤度マップと LiDAR による大域的自己位置推定に関する手法を提案する．まず，前章で述べた Wi-Fi 信号強度を格納した地図を基に，ロボット存在確率を表す Wi-Fi 尤度マップを構築し，このマップを用いた大域的自己位置推定を行う手法について説明する．次に，Wi-Fi 尤度マップの位置推定結果から，LiDAR を用いてロボットの姿勢を推定する手法を述べる．最後に，これらの手法を統合することで，大域的自己位置推定が可能であるか検証する．

4.2　ハードウェア構成

本研究で使用するハードウェア構成について説明する．本研究で使用したロボットは，外形が全幅 0.55[m]，全長 0.72[m]，全高 0.6[m]，全重量 86[kg] の独立 2 輪駆動の車輪型ロボットである．ロボットの前方には 3D-LiDAR が搭載しており，2 次元スキャンデータを用いて幾何地図作成や自己位置推定を行った．3 章では，RS-LiDAR-32(Range: 0.4–200 [m]) を用いて，4 章では，VLP-16(Range: 0.5–100 [m]) を用いて実験を行った．

4.2.1　Wi-Fi センサ

このセンサーは無指向性であり，全方向から Wi-Fi 信号を受信できる．

Fig. 4.1 WiFi sensor

Table 4.1 Robot specification

Robot name	Rebot
Size	Width = 0.55 [m] Depth = 0.72 [m] Height = 0.60 [m]
Weight	86.0 [kg]
PC	Bord：AZW SER CPU : Ryzen 5 5500u (AMD) Mem：16GB OS : Ubuntu 22.04 LTS
IMU	MPU9250 (TDK InvenSense)
3D-LiDAR	RS-LiDAR-32 (robosense) ※ Chapter 3 VLP-16 (Velodyne Lidar) ※ Chapter 4
2D-LiDAR	RPLIDAR A2 ×2 (Slamtec)
Wi-Fi antenna	Dual Band AC1200M Wi-Fi Adapter (Kangzio) ×8
Motor	BLH5100K-A (Oriental motor)
Motor Driver and Encorder	BLVD20KM (Oriental motor)
Battery	IFM24-400E2 ×2 (Kung Long Batteries)

4.3　自己位置推定に用いる地図

LiDAR または Wi-Fi を用いた自己位置推定を行うには，それぞれのセンサごとに対応した地図が必要となる．本研究室におけるそれぞれの地図構築手法について説明する．

4.3.1　幾何地図

幾何地図の作成には，ROS の SLAM パッケージの一つである gmapping を使用した．gmapping は，オドメトリデータと LiDAR の 2 次元スキャンデータを用いて地図を作成するパッケージである．本論文では，LiDAR により取得した 2D スキャンデータと，エンコーダと IMU データから算出したオドメトリ情報を入力として地図を作成した．gmapping により生成される地図は，障害物があるグリッドを黒 (0)，障害物がないグリッドを白 (255)，未知の領域を灰色 (127) の値で出力される．

本研究室で使用する LiDAR による自己位置推定では，幾何地図として占有格子地図を用いるため，gmapping で生成した地図を各格子ごとに障害物の存在確率を表した地図に変換する必要がある．ここで，地図の中で障害物を占有しているグリッド間のユーグリッド距離

Table 4.2 Specification of WiFi sensor

Standard	IEEE 802.11ac, IEEE 802.11n, IEEE 802.11g, IEEE 802.11b
Interface	USB3.0
Frequency Range	2.4GHz 〜 2.4835GHz; 5.15GHz 〜 5.25GHz
Wireless Speed	11ac; up to 867Mbps 11n; up to 300Mbps 11g; 6/9/12/18/24/36/48/54 Mbps 11b; 1/2/5.5/11 Mbps
Working Channel Number	1 〜 14(2.4G); 36 〜 165(5G)
Spread Spectrum	DSSS
Modulation Type	OFDM/CCK/16-QAM/64-QAM
Wireless Transmit Power	20dBm (MAX EIRP)
Operate Environment	Operating Temperature: 0℃ 〜 40℃(32°F) Storage Temperature: -40℃ 〜 70℃(-40°F) Relative Humidity: 10% 〜 90%RH non-condensing Storage Humidity: 5% 〜 90%RH non-condensing

$dist$ を定義したとき，地図上で占有される確率を平均ゼロのガウス分布で以下の式により表す (3.1).

$$m_{u,v} = z_{hit} exp\left(\frac{dist^2}{2\sigma^2}\right) + (1 - z_{hit}) \tag{4.1}$$

ここで，z_{hit} はガウス分布による LiDAR の重みを表し，観測のノイズや外れ値を考慮するためのパラメータである．また $m_{u,v}$ は地図上の位置 u，v における地図の占有確率を示す．σ はガウス分布の分散であり，占有の広がりを決定する要因である．グリッドサイズは 0.2[m] とした.

4.3.2　Wi-Fi 地図

Wi-Fi 地図の地図構築に必要なステップは下記の通りである.

1. データ取得

ロボットを手動走行させ，オドメトリ，LiDAR のスキャンデータ，Wi-Fi 信号強度を記録する．次にオドメトリ，LiDAR のスキャンデータから先ほどの gmapping を用いて幾何地図を作成する．幾何地図作成後，ros の AMCL から幾何地図を作成した際のデータを用いて自己位置推定を行う．その自己位置推定結果 $\mathbf{x} = (x, y, \theta)$ と,Wi-Fi 信号強度の観測値 $\mathbf{z}$ に対応

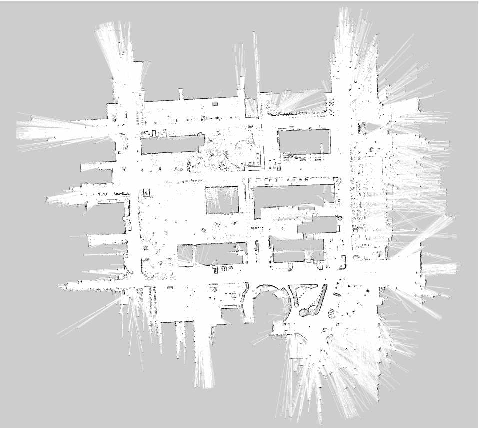


Fig. 4.2 Geometric map created by gmapping

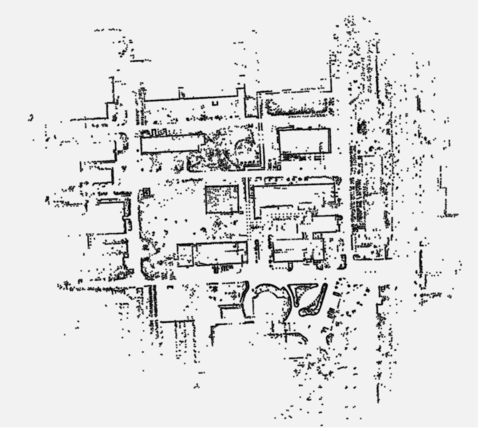


Fig. 4.3 Map converted from geometric map to occupied grid map

付けたデータセットを作成する．ただし，$\mathbf{z}$ は，位置 $\mathbf{x}$ で観測できた異なる m 個の AP の信号強度を並べたベクトル $\mathbf{z} = [z_1, z_2, ..., z_m]$ である．

この際，各マックアドレスの Wi-Fi の信号強度は 3 秒間に観測されたデータの平均値とした．また，5 か所以上で Wi-Fi の信号強度を観測できた AP のみを使用した．これはモバイル AP を含まないようにするためである．ただしこれらの数値は，Wi-Fi の信号強度のノイズや減衰の状況などを考慮して決定したものではないため，より適切な数値の選択のための手法構築は今後の課題である．

2. ガウス回帰過程

ロボットがデータ取得時に走行した経路上のみ Wi-Fi 信号強度のデータが存在し，走行経路以外ではデータがないため，自己位置推定が不可能である．したがって，任意の経路で自己位置推定を行うためには，走行経路全域の Wi-Fi 信号強度を格納した地図が必要となる．そこで本研究室では，ガウス過程回帰 (GPR:Gaussian Process Regression) を用いて，データセットをもとに走行経路外の Wi-Fi の信号強度を補完することで，走行環境全域の信号強度の分布を予測した [**miyagusuku2018precise**].

3. Wi-Fi 地図作成

ガウス過程回帰により取得した分布をもとに，指定した解像度の格子にデータを格納することで，最終的な Wi-Fi 信号強度を用いた自己位置推定に用いる地図が完成する．予測値の平均値は各 AP ごとに行われ，分散は全ての AP に共通するため，Wi-Fi 地図はアクセスポイント数枚，分散地図は一枚となる．グリッドサイズは 1[m] とした．

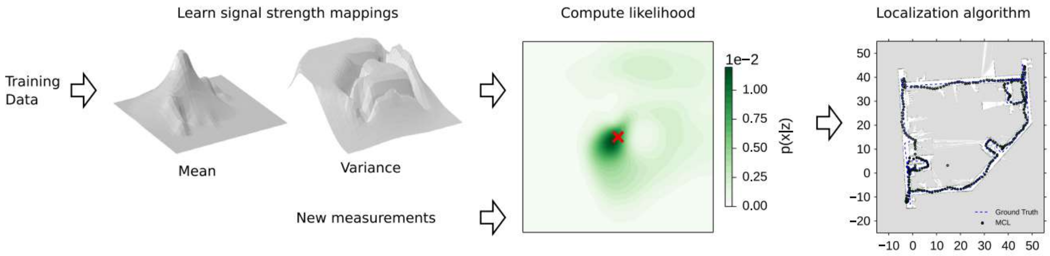

Learn signal strength mappings
Training Data
Mean
Variance
Compute likelihood
New measurements
1e−2
1.00
0.75
0.50
0.25
0.00
p(x|z)
Localization algorithm
50
40
30
20
10
0
−10
−20
−10
0
10
20
30
40
50
Ground Truth
MCL

Fig. 4.4 Overview of wireless-based localization using signal strength mapping[**miyagusuku2018precise**]

4.4　LiDAR を用いた局所的自己位置推定

本研究室の LiDAR を用いた自己位置推定は Dellaert らによって提案された MCL[**mcl**] がベースとなっている．MCL とは，ロボットを疑似に表現したパーティクルと呼ばれる粒子を用いることで，ロボットの存在確率を表現する手法である．位置推定のプロセスは以下の通りである．まず，ロボットの初期位置付近にパーティクルをサンプリングする．ここで，各パーティクルはロボットの状態の一つの候補を表す．次に，ロボットが移動した際，内界センサによる移動量から動作モデルに従ってパーティクルの状態が更新される．その後に，ロボットが外界センサにより観測されたセンサデータを各パーティクルに与える．ここで，パーティクルが観測するであろうセンサデータと実際に観測したセンサデータとの比較を行い，一致している度合いに応じて各パーティクルに尤度を与える．この尤度が大きいほどロボットの状態に近いとされ，ロボットの位置推定に利用される．さらに，パーティクルの尤度の偏りを検知する Effective Sample Size(ESS)[**ess**] を用いることで，全パーティクルの中で自己位置推定に有効なパーティクルの個数を検知する．この ESS が一定の閾値を下回った場合にパーティクルにリサンプリングを行う．このとき，尤度の小さいパーティクルは削除され，尤度の高いパーティクルにサンプリングされる．以上のプロセスを一度行うと，自己位置推定が 1 ステップ実行されたことになる．ロボットはこのプロセスを繰り返すことで，自己位置を確信する．パーティクルは確率分布に基づいてサンプリングされるため，ミスマッチングによるパーティクルの収束を防げる可能性がある．そのため，ロバストな自己位置推定の実現が可能になる．パーティクル数が増加すれば，それに伴ってロボットの存在確率をより近似できるためロバスト性が向上する．しかし，パーティクルの個数が多すぎると情報量も増加するため，計算コストが高くなってしまい，リアルタイムに位置推定が行えなくなるといった課題もある．以下に本手法の位置推定の流れを示す．

4.4.1　動作モデルに基づいたパーティクルの更新

動作モデルを考える時，時刻 $t-1$ の時点でパーティクルが与えられていると仮定する．ロボットは車輪の回転量から時刻 $t-1$ から時刻 t の間の移動量を計測することができる．この移動量を用いて，ロボットの動作モデルに従って状態を更新すると，時刻 t における事前分布を予測できる．この予測をオドメトリと呼び，車輪型移動ロボットにおいて，広く用いられる [**fox2005probabilistic**]．以下でオドメトリによる動作モデルについて説明をする．

本研究で対象とする車輪型ロボットは，二次元平面上でのみを移動すると仮定する．

$$\mathbf{x} = \begin{pmatrix} x \\ y \\ \theta \end{pmatrix} \tag{4.2}$$

で表現される．また，ロボットの前回 $\mathbf{x}_{t-1}$ と現在の状態 $\mathbf{x}_t$ までの動作を表す 3 つのパラメータ $\hat{\delta}_{rot1}$，$\hat{\delta}_{trans}$，$\hat{\delta}_{rot2}$ を次のように定める．

$$\hat{\delta}_{rot1} = \delta_{rot1} + \varepsilon_{\alpha 1}\delta_{rot1} + \alpha_2\delta_{trans} \tag{4.3}$$

$$\hat{\delta}_{trans} = \delta_{trans} + \varepsilon_{\alpha 3}\delta_{trans} + \alpha_4\delta_{rot1} + \alpha_4\delta_{rot2} \tag{4.4}$$

$$\hat{\delta}_{rot2} = \delta_{rot2} + \varepsilon_{\alpha 1}\delta_{rot2} + \alpha_2\delta_{trans} \tag{4.5}$$

ここで，ε_b は平均 0，b の正規分布に従う値とし，式 (3.3)〜式 (3.5) 内に b の実体として付されている α_1〜α_4 はロボット固有のノイズパラメータである．走行距離ノイズを σ_{dist}，角度ノイズを σ_{angle} として，これらのノイズパラメータを以下のように定義する．

α_1：回転運動による回転誤差 $\alpha_1 = \sigma_{\mathrm{angle}}$
α_2：並進運動による回転誤差 $\alpha_2 = \sigma_{\mathrm{angle}} \times \sigma_{\mathrm{dist}}$
α_3：並進運動による並進誤差 $\alpha_3 = \sigma_{\mathrm{dist}}$
α_4：回転運動による並進誤差 $\alpha_4 = \sigma_{\mathrm{dist}} \times \sigma_{\mathrm{angle}}$

以上より，$\hat{\delta}_{rot1}$，$\hat{\delta}_{trans}$，$\hat{\delta}_{rot2}$ を用いたオドメトリ動作モデルは，次式として表現できる．

$$\begin{pmatrix} x_{t+1} \\ y_{t+1} \\ \theta_{t+1} \end{pmatrix} = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} + \begin{pmatrix} \hat{\delta}_{trans}\cos(\theta_t + \hat{\delta}_{rot1}) \\ \hat{\delta}_{trans}\sin(\theta_t + \hat{\delta}_{rot1}) \\ \theta_t + \hat{\delta}_{rot1} + \hat{\delta}_{rot2} \end{pmatrix} \tag{4.6}$$

これにより，パーティクルが移動の不確かさを表現することにより，パーティクルの分布が拡張して遷移していく．

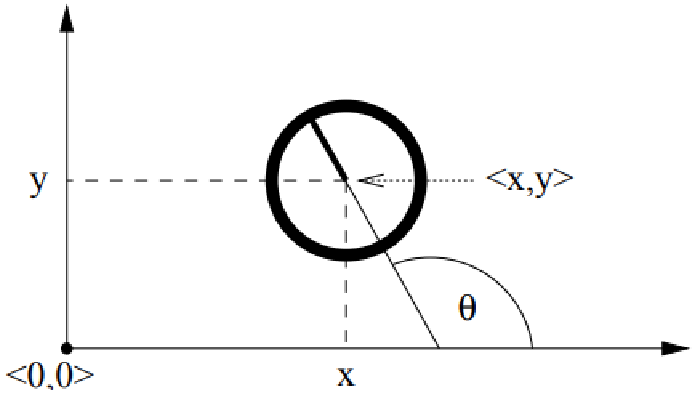


Fig. 4.5 Robot Pose Shown in a Global Coordinate System[**fox2005probabilistic**]

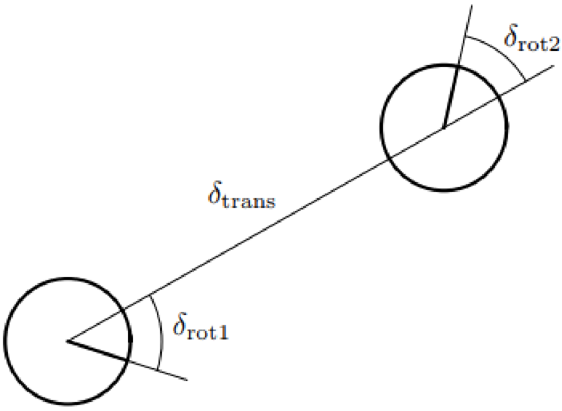


Fig. 4.6 Odometry model: The robot motion in the time interval$(t-1, t)$ is approximated by a rotation δ_{rot1}, followed by a translation δ_{trans} and a second rotation δ_{rot2}. The turns and translation are noisy[**fox2005probabilistic**].

4.4.2　尤度計算

尤度計算は，動作モデルにより予測された各パーティクルの尤もらしさを算出するステップである．尤もらしさとは確率に似た概念であるが，確率の定義である和が 1 になるという制約を満たさないため，確率とは別の呼び方をしている．

本研究室では，幾何地図として占有格子地図を用いる．占有格子地図は 255 階調のグレースケールのグリッドマップで表現している．そのため，観測の独立性を仮定することで，パーティクルの尤度 ω を次のように求める．

$$\omega_i = \frac{m_{u,v}}{b} \tag{4.7}$$

$$\omega = exp\left\{\frac{1}{N}\sum_{i=1}^{N}(log\ \omega_i)\right\} \tag{4.8}$$

ここで，LiDAR のスキャンデータの位置 u，v における地図の占有確率を $m_{u,v}$，LiDAR のレーザ 1 本に対する尤度を ω_i，LiDAR の 1 回のスキャンで観測したレーザの本数を N，輝度の最大値を b とする [**onolidar**]．この尤度計算は地図に表現されている情報を尤度として捉えて設計された尤度関数である．この尤度関数は他の LiDAR の尤度関数に比べて計算コストが低いため，リアルタイムな位置推定が求められるロボットの尤度計算に適している．しかし，計測値を短くする原因となる動的障害物の影響を考慮した設計とはなっていないため，より動的障害物に強い手法の構築は今後の課題である．

4.4.3　位置推定

MCL はロボットの位置を確率分布として表現するフレームワークであるが，ロボットのナビゲーションのためには確率分布ではなく，1 点の代表点を計算する必要がある．代表点の計算手法として，最尤値計算と重み付け平均の 2 種類あるが，本論文では重み付け平均を採用する．したがって，式 (3.9) に従って時刻 t におけるロボットの推定結果とする．j はある地点でのパーティクルで，n はパーティクル数を表している．

$$\mathbf{x}_t = \sum_{j=1}^{n} \omega_t^j x_t^j \tag{4.9}$$

4.4.4　パーティクルのリサンプリング

MCL では 3.4.1 節で説明した通り，1 ステップ移動するたびにノイズをモデル化した確率分布からパーティクルをサンプリングするため，パーティクルによってはロボットの真の状態と大きく異なっている可能性がある．この場合，予測された確率分布は広がり続けてしまい位置推定性能が低下する．したがって，不要なパーティクルは削除し，真の状態に近い場所にサンプリングし直すことで，より精密かつ正確な確率分布を生成したい．このときパーティクルをサンプリングし直すことをリサンプリングとよぶ．リサンプリングは，パーティクルの重みに偏りが現れた際にのみに行う方が効率がよい．尤度の偏り検知には式 (3.10) により計算される ESS を用いた．これは全パーティクルの中で自己位置推定に有効なパーティクルの個数を検知する．この ESS が一定の閾値を下回ったときパーティクルのリサンプリングを行う．

$$\eta_{ess} = \frac{1}{\sum_{j=1}^{n} \omega_j^2} \tag{4.10}$$

第 5 章

ソフトウェア設計

5.1 カルマンフィルタの実装

カルマンフィルタの疑似コードは 1 です．あああ

Algorithm 1 Kalman Filter

```
Input: Initial State $\hat{\mathbf{x}}_{0|0}$, Initial Covariance $\mathbf{P}_{0|0}$
Output: Estimated State at time $k$, $\hat{\mathbf{x}}_{k|k}$

for $k = 1, 2, \ldots$ do
    Prediction Step
        $\hat{\mathbf{x}}_{k|k-1} \leftarrow \mathbf{A}\hat{\mathbf{x}}_{k-1|k-1} + \mathbf{B}\mathbf{u}_k$    ▷ Project the state ahead
        $\mathbf{P}_{k|k-1} \leftarrow \mathbf{A}\mathbf{P}_{k-1|k-1}\mathbf{A}^\top + \mathbf{Q}$    ▷ Project the error covariance ahead

    Update Step
        $\mathbf{K}_k \leftarrow \mathbf{P}_{k|k-1}\mathbf{C}^\top(\mathbf{C}\mathbf{P}_{k|k-1}\mathbf{C}^\top + \mathbf{R})^{-1}$    ▷ Compute the Kalman gain
        $\hat{\mathbf{x}}_{k|k} \leftarrow \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k(\mathbf{y}_k - \mathbf{C}\hat{\mathbf{x}}_{k|k-1})$    ▷ Update the state estimate
        $\mathbf{P}_{k|k} \leftarrow (\mathbf{I} - \mathbf{K}_k\mathbf{C})\mathbf{P}_{k|k-1}$    ▷ Update the error covariance
end for
```

5.2 ソフトウェア評価

ソフトウェアの評価をします．いいい

いいい

第 6 章

実環境におけるシステム評価

6.1　実験目的

(6.1) はガウス分布です.

$$f(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{6.1}$$

6.2　実験環境

実験環境を説明します.

6.3　実験結果

ロボットのオドメトリは (6.2) で計算されます．次の位置姿勢 $(x_{k+1}, y_{k+1}, \theta_{k+1})$ は，現在の位置姿勢 (x_k, y_k, θ_k) と，微小時間 Δt の間の並進速度 v および角速度 ω を使って更新されます.

$$\begin{cases} x_{k+1} = x_k + v\Delta t \cos(\theta_k + \frac{\omega\Delta t}{2}) \\ y_{k+1} = y_k + v\Delta t \sin(\theta_k + \frac{\omega\Delta t}{2}) \\ \theta_{k+1} = \theta_k + \omega\Delta t \end{cases} \tag{6.2}$$

6.3.1　現在の位置姿勢

(x_k, y_k, θ_k) は現在の位置姿勢です.

6.3.2　次の位置姿勢

$(x_{k+1}, y_{k+1}, \theta_{k+1})$ 次の位置姿勢です.

6.3.3　更新

微小時間 Δt の間の並進速度 v および角速度 ω を使って更新されます.

Table 6.1 ICP によって算出された補正量

データ	x [m]	y [m]	z [m]	Roll [deg]	Pitch [deg]	Yaw [deg]	Score
1	0.033	-0.011	-0.036	0.982	-0.336	0.491	2.454
2	0.033	-0.008	-0.038	1.014	-0.510	0.456	2.273
3	0.031	-0.010	-0.035	0.935	-0.226	0.510	2.119
4	0.029	-0.007	-0.035	0.946	-0.264	0.518	2.380
5	0.017	0.003	-0.034	0.999	-0.233	0.441	2.186
6	0.025	0.000	-0.039	0.938	-0.469	0.473	2.289
7	0.017	0.001	-0.034	0.926	-0.184	0.422	2.303

Table 6.2 ICP 補正量の統計値

項目	平均	最小値	最大値	標準偏差
x [m]	0.0264	0.017	0.033	0.0065
y [m]	-0.0046	-0.011	0.003	0.0053
z [m]	-0.0359	-0.039	-0.034	0.0018
Roll [deg]	0.963	0.926	1.014	0.032
Pitch [deg]	-0.317	-0.510	-0.184	0.117
Yaw [deg]	0.473	0.422	0.518	0.033
Score	2.286	2.119	2.454	0.104

6.3.4　2 台の 3D-LiDAR 間の位置姿勢補正

2 台の 3D-LiDAR 間の位置・姿勢ずれを補正するため，LiDAR 1 の点群を基準として LiDAR 2 の点群に ICP を適用した．7 フレームについて得られた ICP 補正量を table 6.1 に示す．

ICP によって得られた補正量の統計値を table 6.2 に示す．

並進補正量の大きさは平均 45.3 mm，最小 38.0 mm，最大 51.0 mm であった．また，回転補正量の大きさは平均 1.12 deg，最大 1.22 deg であった．

これらの結果をもとに，LiDAR 2 に適用する固定補正値を table 6.3 のように設定した．

Table 6.3 LiDAR 2 に適用する固定補正値

x [m]	y [m]	z [m]	Roll [deg]	Pitch [deg]	Yaw [deg]
0.026	-0.006	-0.034	0.95	-0.20	0.47

Table 6.4 固定補正適用後の ICP 残差

データ	x [m]	y [m]	z [m]	Roll [deg]	Pitch [deg]	Yaw [deg]	Score
1	0.001	0.000	-0.001	-0.011	-0.018	0.012	2.376
2	-0.003	0.004	0.000	-0.016	0.064	0.039	2.341
3	0.004	-0.001	-0.001	0.049	-0.109	0.023	2.355
4	0.006	0.001	-0.004	0.089	-0.315	0.003	2.299
5	0.007	-0.005	-0.002	0.031	-0.148	0.025	2.331
6	0.000	0.000	0.000	-0.004	0.011	0.005	2.118
7	0.001	-0.001	0.000	-0.010	0.010	0.028	2.300

Table 6.5 固定補正後の残差

項目	平均残差	平均絶対残差	最大絶対残差
x [m]	0.0023	0.0031	0.007
y [m]	-0.0003	0.0017	0.005
z [m]	-0.0011	0.0011	0.004
Roll [deg]	0.018	0.030	0.089
Pitch [deg]	-0.072	0.096	0.315
Yaw [deg]	0.019	0.019	0.039

固定補正値を適用した後，再度 ICP を用いて残差を評価した．結果を table 6.4 に示す．

固定補正後の残差の統計値を table 6.5 に示す．

3 次元の並進残差の大きさは平均 4.0 mm，最大 8.8 mm であった．以上より，ICP から算出した補正値を固定値として適用することで，2 台の 3D-LiDAR 間の点群ずれを 1 cm 以内まで低減できることを確認した．

6.4　考察

考察します．

第 7 章

シミュレーション評価

7.1　シミュレーション環境の構築

環境をつくりました．あああ

7.2　実験概要

シミュレーションします．いい

7.3　考察

考察を 7.1 にまとめます．うう

Table 7.1 性能比較

項目	提案手法	従来手法 A	従来手法 B
精度 (%)	95.2	92.1	88.5
処理時間 (ms)	12.5	18.0	10.2
消費電力 (W)	3.1	3.5	2.9

第 8 章

結言

8.1 結論

結論を述べます.

8.2 課題

課題を述べます.

8.3 展望

展望を述べます.

謝辞

先生に感謝します．ああ

共同研究者に感謝します．いい

研究室メンバーに感謝します．うう

家族に感謝します．ええ

結びます．おおお

著者氏名

参考文献

[1]厚生労働省. “令和 7 年版 労働経済の分析ー労働力供給制約の下での持続的な経済成長に向けてー.” [Online; accessed 16-October-2026]. [Online]. Available: `https://www.mhlw.go.jp/stf/wp/hakusyo/roudou/25/2.html` (cit. on p. 1).

[2]K. Ozaki, H. Asama, et al., “Negotiation method for collaborating team organization among multiple robots,” in *Distributed Autonomous Robotic Systems*, H. Asama, T. Fukuda, et al., Eds., Tokyo: Springer Japan, 1994, pp. 199–210 (cit. on p. 4).

[3]R. Miyagusuku, K. Tabata, et al., “Gaussian gridmaps from gaussian processes for wifi-based robot self-localization in outdoor environments,” in *2025 IEEE/SICE International Symposium on System Integration (SII)*, 2025, pp. 1593–1598 (cit. on p. 4).

[4]K. Tabata, H. Seki, et al., “Realization of swing manipulation by 3-dof robot arm for unknown string via parameter estimation and motion generation,” *International Journal of Automation Technology*, vol. 15, no. 6, pp. 774–783, 2021 (cit. on p. 4).

付録 A

詳細な電子回路図

電子回路図を A.1 に示します．ああ

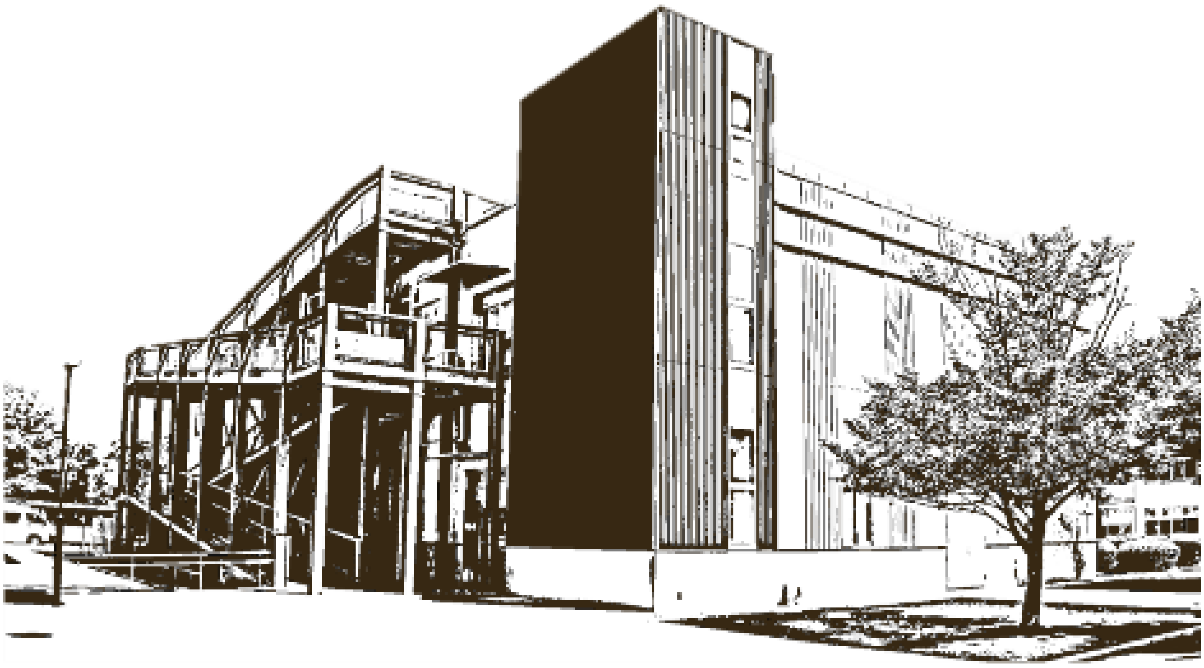


Fig. A.1 システム全体の回路図

付録 B

主要なプログラムのソースコード

B.1 C++

C++ソースコードを B.1 に示します．ああ．

Listing B.1 Hello World Program in C++

```
// C++ source code here.
#include <iostream>

int main() {
  std::cout << "Hello,␣World!" << std::endl;
  return 0;
}
```

B.2 Python

Python ソースコードを B.2 に示します．いい．

Listing B.2 Hello World Program in Python

```
# Python source code here.
# The simplest way to print a line in Python:

def main():
  print("Hello,␣World!")

if __name__ == "__main__":
  main()
```

付録 C

追加実験

C.1　実験 A

　ああ

C.2　実験 B

　いいい